Advanced Programming

Week No. 1

1. Console Application:

A console application, in the context of C#, is an application that takes input and displays output at a command line console with access to three basic data streams: standard input, standard output and standard error.

A console application facilitates the reading and writing of characters from a console - either individually or as an entire line. It is the simplest form of a C# program and is typically invoked from the Windows command prompt. A console application usually exists in the form of a stand-alone executable file with minimal or no graphical user interface (GUI).

Software required for completing this tutorial successfully

In order to successfully complete this tutorial, Microsoft Visual Studio 2010 is required to be installed on your PCs

In today's session

- ✓ What are Console Applications
- ✓ Steps to develop a Console Applications
- ✓ Input/output Data in Console Applications
- ✓ Decision Making in Console Applications

Creating our first Console Application

Use the following steps to develop the Console Application in C-Sharp

1. Open Visual Studio

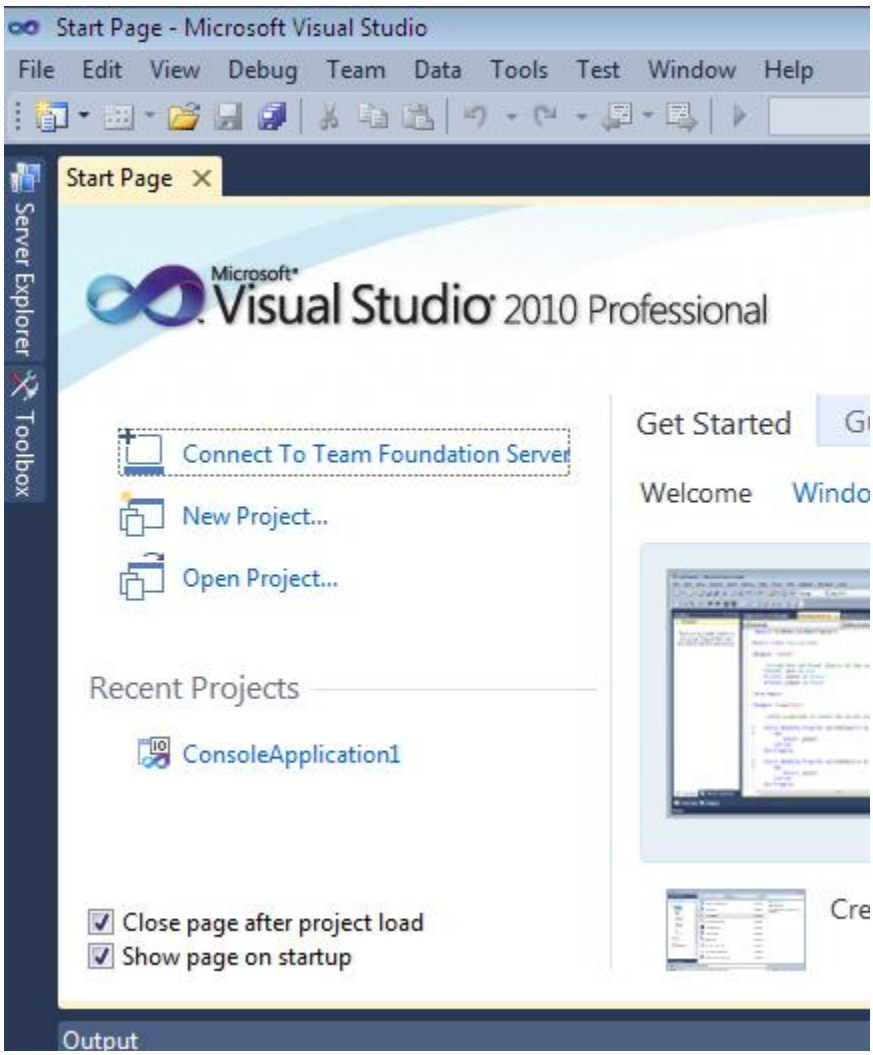

Figure 1

2. Next, Choose File ➔ New ➔ Project as shown

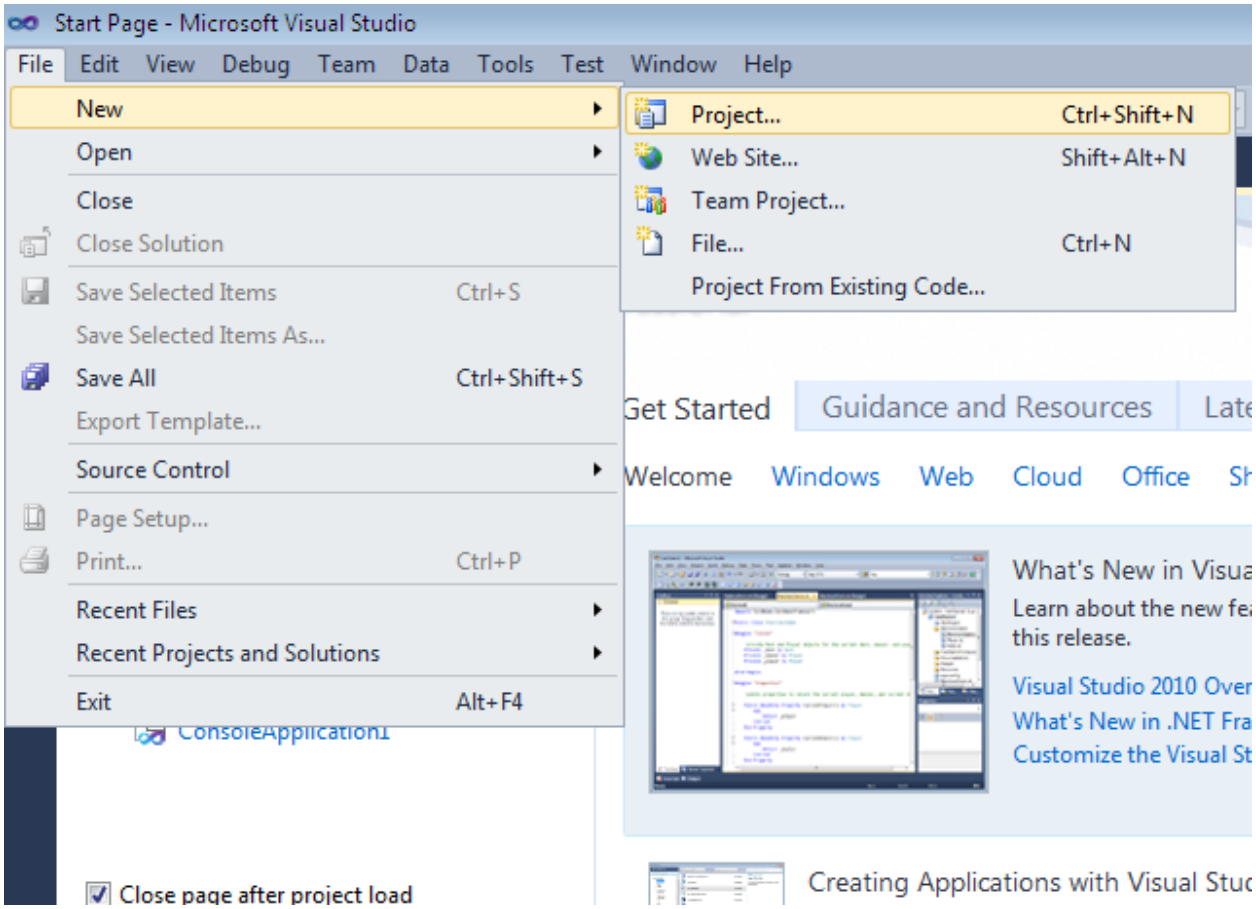

Figure 2

3. Choose visual C# language in the Left hand pane of the open Window as shown in the next figure

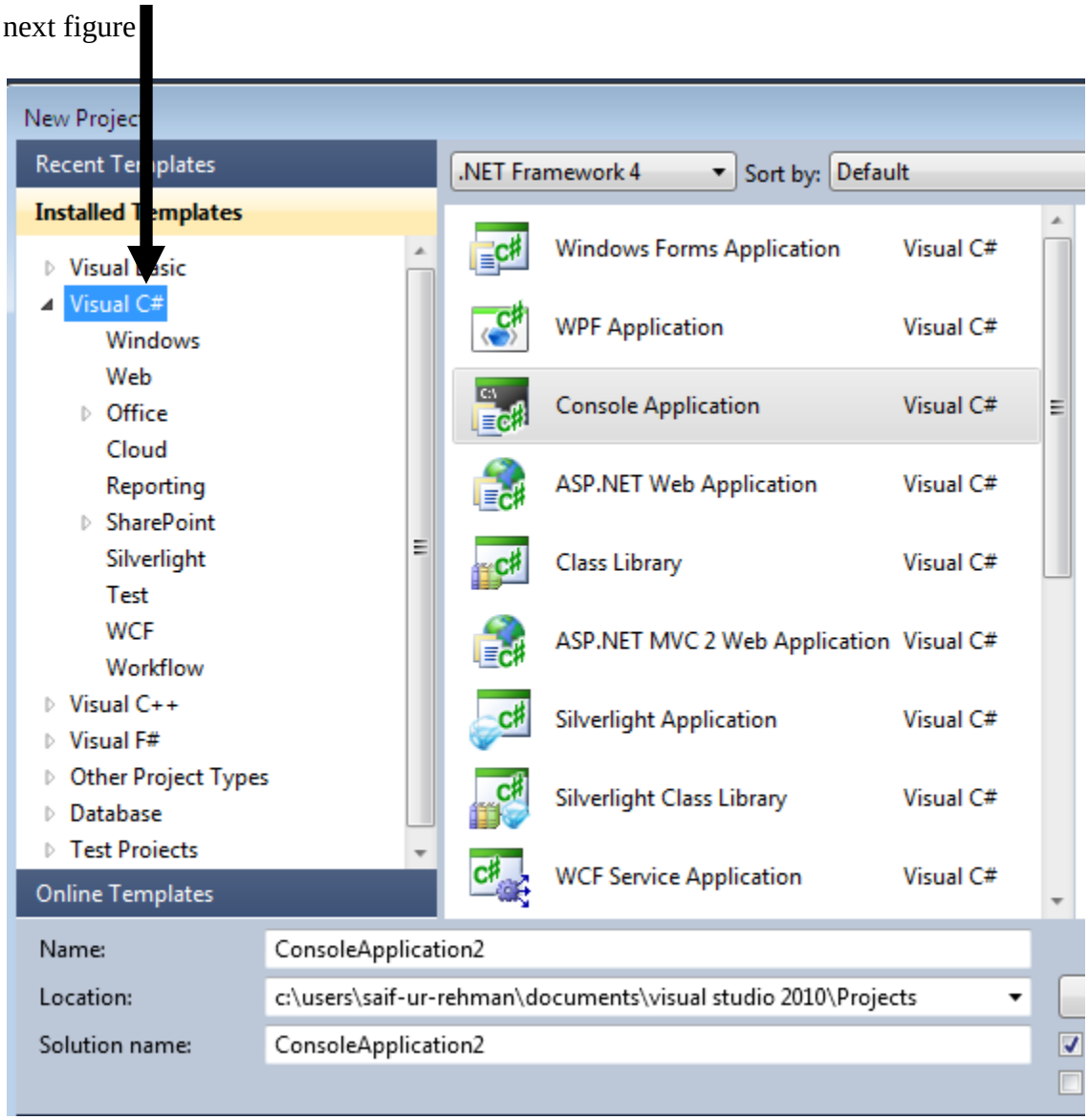

Figure 3

4. Next, Choose **<u>Console Application</u>** from the Right Hand Pane in the Window shown above. Enter the name of the Application as **<u>Week01</u>** as shown below and press the OK Button

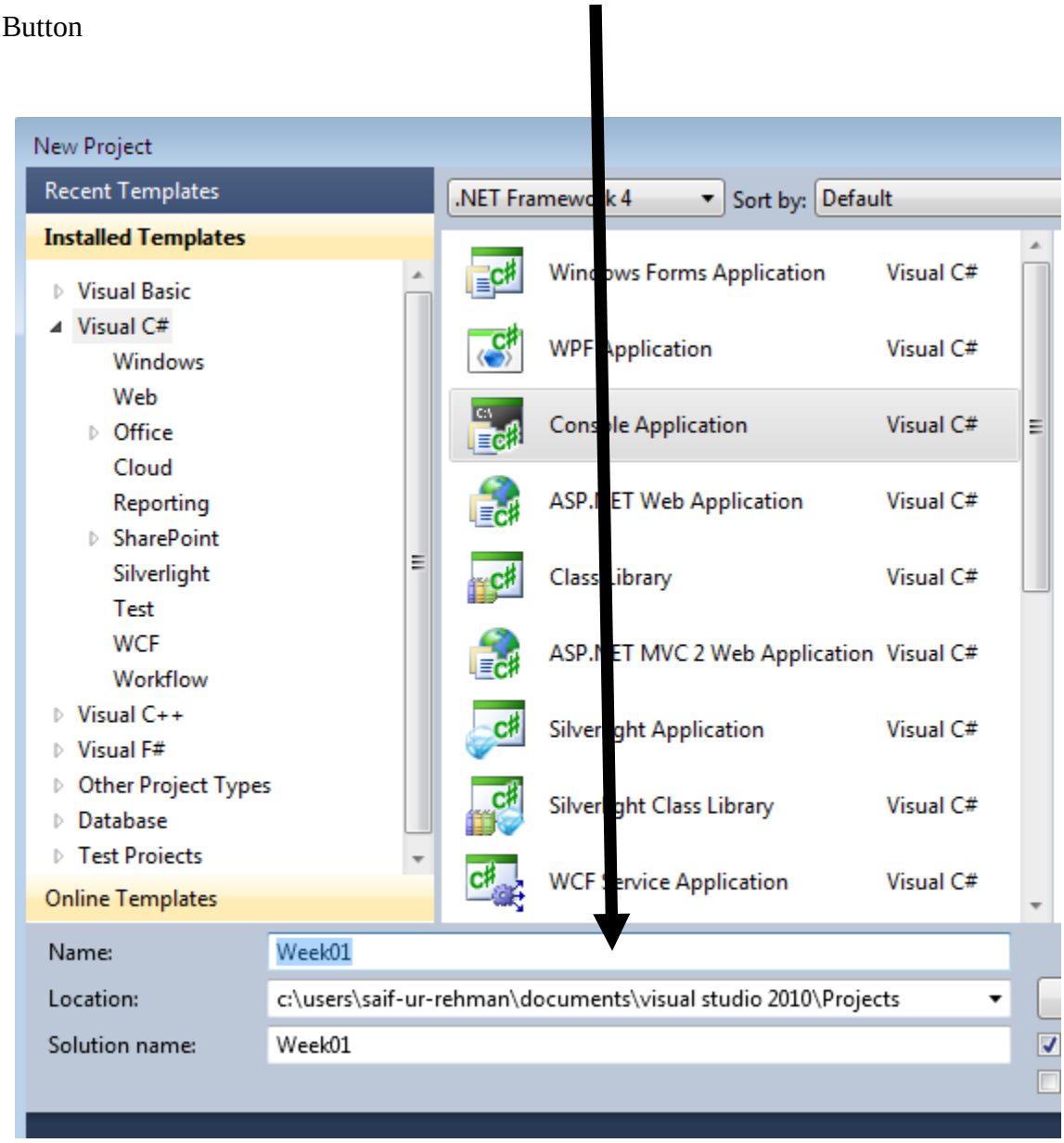

Figure 4

5. You will get the Source Code window as shown below

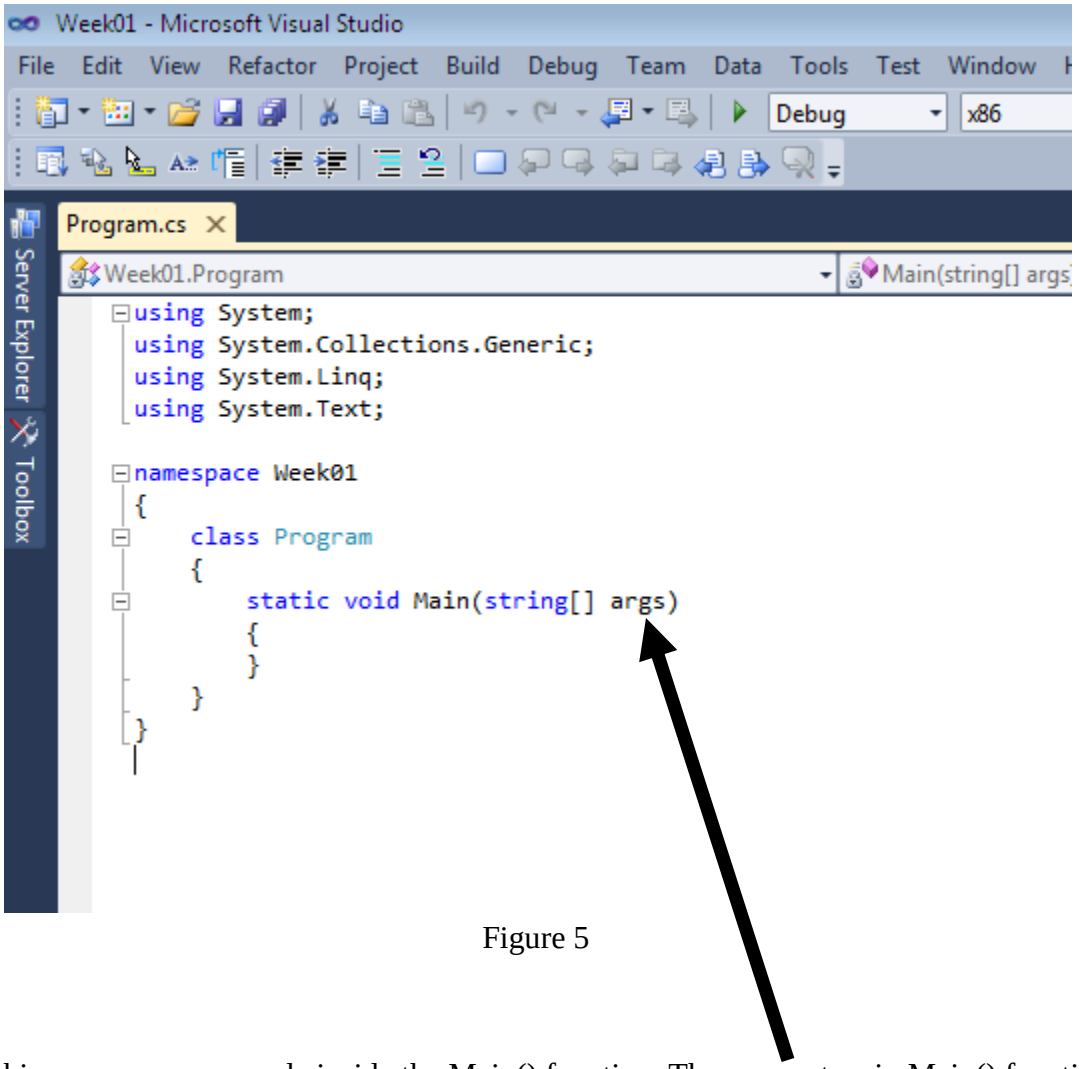

Figure 5

6. In this screen you can code inside the Main() function. The parameters in Main() function are called command line parameters. Values to these parameter can be passed from the command line execution of the program. We will see shortly, how to use these command line arguments. Here, we will see how to use different input/output statements in C#.

   Console.Write() : This method is used to output the text on the screen. This method is functions similar to cout<< or printf() in C/C++ language or System.out,println() method in Java. A simple example of using Console.Write() is shown here

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

namespace Week01
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.Write("Aslamo Alaikum!");

            Console.Write("We Wel Come you to Visual Programming Course");
        }
    }
}
```

Figure 6

7. Compile the application from Build➔Build Solution as shown, if no error found your compilation will be successful and this will show a successful message.

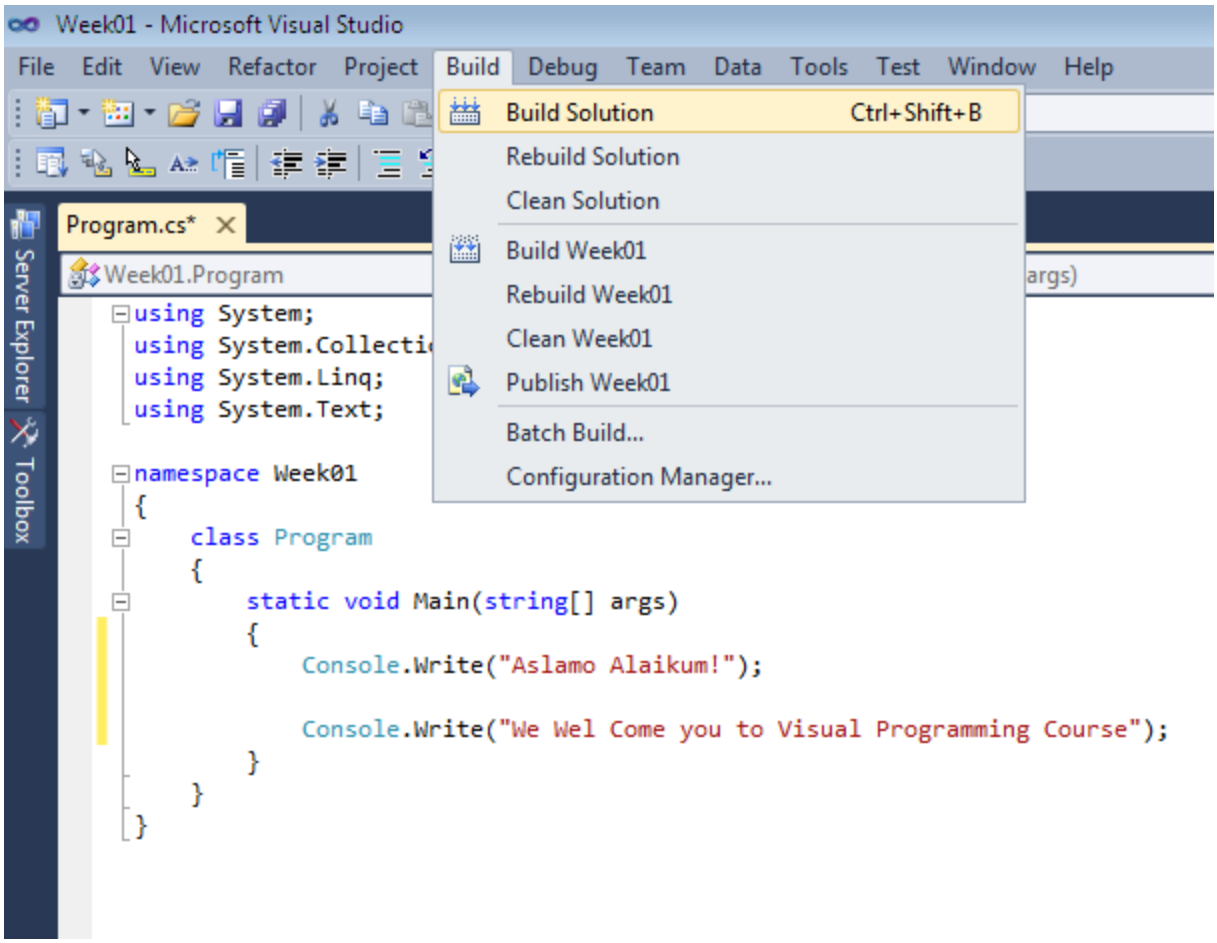

Figure 7

8. After a successful compilation, run the program as shown

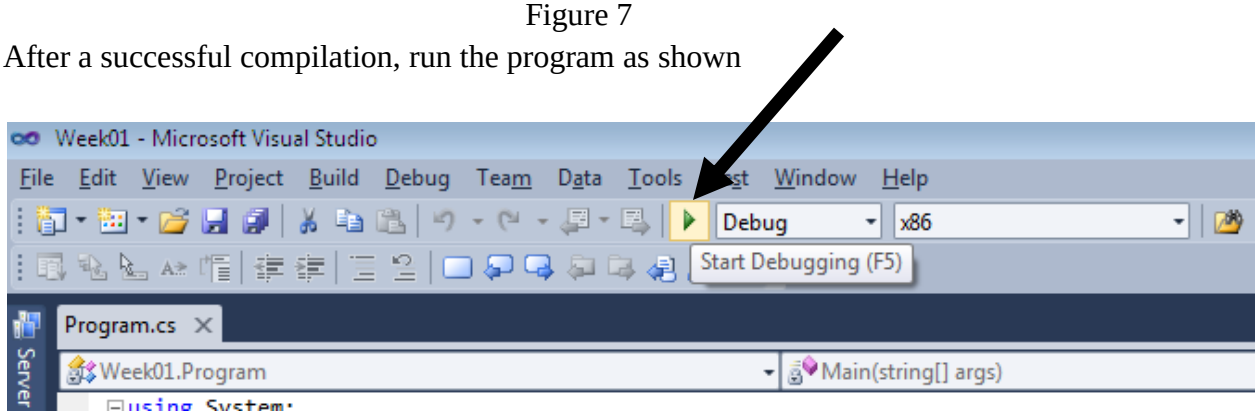

Figure 8

9. You will notice, that program will run, output will disappear soon before you see it. To stop the output for some time or before user press a key from keyboard. Add this line before the closing brace of the Main(), as shown in next figure.

Console.ReadKey();

```
Program.cs
Week01.Program                                  Main(string[] args)
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

namespace Week01
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.Write("Aslamo Alaikum!");

            Console.Write("We Wel Come you to Visual Programming Course");

            Console.ReadKey();
        }
    }
}
```

Figure 9

10. Again Run the Application

```
file:///c:/users/saif-ur-rehman/documents/visual studio 2010/Projects/Week01/Week01/bin/Debug...
Aslamo Alaikum!We Wel Come you to Visual Programming Course
```

Figure 10

11. Next, you can add a line break after Console.Writ() method call by different methods. You can add a line break by **\n** and **Console.WritLine()** to see the line break in action

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

namespace Week01
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.Write("Aslamo Alaikum!\n");

            Console.WriteLine("We Wel Come you to Visual Programming Course");

            Console.ReadKey();
        }
    }
}
```

Figure 11

12. Compile and run the Application again. You will see the line break

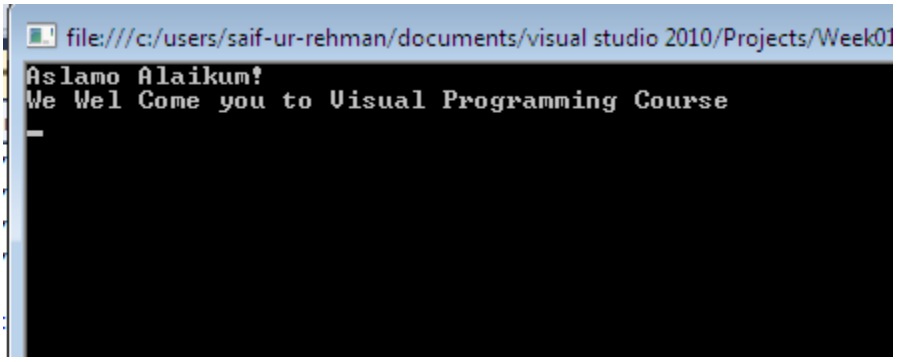

Figure 12

Input in Console Application

We can use different built-in methods to read inputs from users. For example, **Console.ReadLine()** method. This method is used to read the input in the textual format and store in String type of Variables, this is because when data is read using **Console.ReadLine()** method , it is treated as String.

Here is an example, which is showing the usage of **Console.ReadLine().** Code is shown below

```
using System;

namespace ConsoleApplication1
{
    class Program
    {
        static void Main(string[] args)
        {
            String stdName, stdClass, stdSemester;
            Console.WriteLine("Please Enter your Name");
            stdName     = Console.ReadLine();

            Console.WriteLine("Please Enter your Class");
            stdClass    = Console.ReadLine();

            Console.WriteLine("Please Enter your Semester");
            stdSemester = Console.ReadLine();

            Console.WriteLine(); // For Blank Line

            Console.WriteLine("****** Student Details ******");
            Console.WriteLine("Name :" + stdName);
            Console.WriteLine("Class :" + stdClass);
            Console.WriteLine("Semester :" + stdSemester);

            Console.ReadKey();

        }
    }
}
```

Figure 13

Compile and Run the application. The output is shown in the next figure

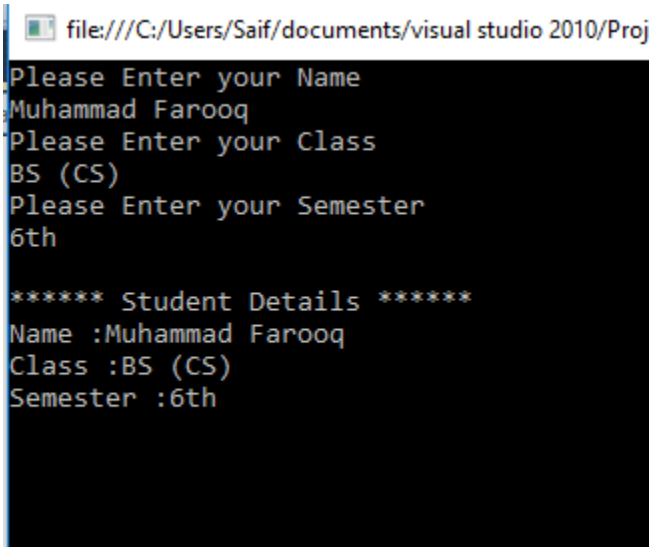

Figure 14

Reading Numeric Input in Console Application

When we read the input using **Console.ReadLine()** method, it is in String format. Furthermore, we can convert it into different types which are desired. Here is an example which shows how to convert the input read by **Console.ReadLine()** into numeric values.

```
using System;

namespace ConsoleApplication1
{
    class Program
    {
        static void Main(string[] args)
        {
            double obtMarks, totMarks;

            Console.WriteLine("Please Enter obtained Marks in Last Semester");
            obtMarks    = Convert.ToDouble(Console.ReadLine());

            Console.WriteLine("Please Enter total Marks in Last Semester");
            totMarks    = Convert.ToDouble(Console.ReadLine());

            double percentage = (obtMarks * 100.0) / totMarks;
```


```
            double percentage = (obtMarks * 100.0) / totMarks;
```


```
            Console.WriteLine(); // For Blank Line

            Console.WriteLine("****** Student Percentage ******");
            Console.WriteLine("Obtained Marks in Last Semester :" + obtMarks);
            Console.WriteLine("Total Marks in Last Semester :" + totMarks);
            Console.WriteLine("Percentage in Last Semester :" + percentage);


            Console.ReadKey();

        }
    }
}
```

Figure 15

Here, we can see addition of new method **Convert.ToDouble()**, this method is used to convert the string input value into floating type value. By executing the application, you can see the output as shown in the next figure below.

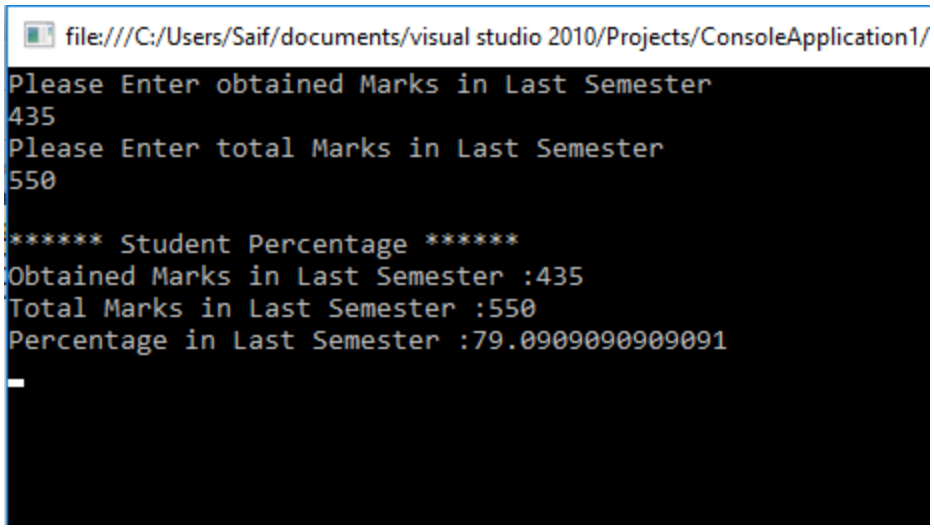

Figure 16

Similarly, there are other methods are also available for converting the data read by **Console.ReadLine()** to different desired data types including integer.

Convert.ToInt16() used to convert the data input by **Console.ReadLine()** to an equivalent 16-bits signed integer

Convert.ToInt32() used to convert the data input by **Console.ReadLine()** to an equivalent 32-bits signed integer

Convert.ToInt64() used to convert the data input by **Console.ReadLine()** to an equivalent 64-bits signed integer

Decision Making in Console Application

1. if statement

if statement is similar in C# as it is in C , C++ or Java. A simple example below show its usage

```
using System;

namespace ConsoleApplication1
{
    class Program
    {
        static void Main(string[] args)
        {
            int inputNumber;

            Console.WriteLine("Please Enter a Number");
            inputNumber    = Convert.ToInt16(Console.ReadLine());

          if(inputNumber % 2 == 0)
            Console.WriteLine("Input Number " + inputNumber + " Is Even One...");

            Console.ReadKey();

            }
        }
    }
```

Figure 17

2. if-else statement

if-else statement is similar in C# as it is in C, C++ or Java. A simple example below show its usage

```
using System;

namespace ConsoleApplication1
{
    class Program
    {
        static void Main(string[] args)
        {
            int inputNumber;

            Console.WriteLine("Please Enter a Number");
            inputNumber    = Convert.ToInt16(Console.ReadLine());

          if(inputNumber % 2 == 0)
            Console.WriteLine("Input Number " + inputNumber + " Is Even One...");
          else
              Console.WriteLine("Input Number " + inputNumber + " Is Odd One...");
            Console.ReadKey();

        }
    }
}
```

Figure 18

3. Nested if-else statement

Nested if-else statement is similar in C# as it is in C , C++ or Java. A simple example below show its usage

```
using System;

namespace ConsoleApplication1
{
    class Program
    {
        static void Main(string[] args)
        {
            int inputNumber;

            Console.WriteLine("Please Enter a Number");
            inputNumber    = Convert.ToInt16(Console.ReadLine());


          if(inputNumber > 0)
            Console.WriteLine("Input Number " + inputNumber + " Is Positive One...");
          else if(inputNumber < 0)
              Console.WriteLine("Input Number " + inputNumber + " Is Negative One...");
          else
              Console.WriteLine("Input Number " + inputNumber + " Is Zero...");
            Console.ReadKey();

        }
    }
}
```

Figure 19

4. switch statement

switch statement is similar in C# as it is in C , C++ or Java. A simple example below show its usage

```
using System;

namespace ConsoleApplication1
{
    class Program
    {
        static void Main(string[] args)
        {
            char inputCharacter;

            Console.WriteLine("Please Enter a Character [A-Z]....");
            inputCharacter =  Convert.ToChar(Console.ReadLine());

            switch (inputCharacter)
            {
                case 'A':
                case 'E':
                case 'I':
                case 'O':
                case 'U':
                    Console.WriteLine("Vowel character entered.....");
                    break;
                default:
                    Console.WriteLine("Invaild Character entered.....");
                    break;
            }
            Console.ReadKey();

        }
    }
}
```

Figure 20

Congratulations! We Have successfully Completed today's Tutorial

Visual Hands on are prepared by

Saif Ur Rehman, Assistant Professor

UIIT, PMAS Arid Agriculture University, Rawalpindi